

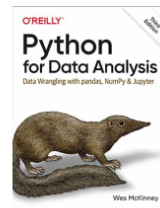
07.2: Data Wrangling and Cleaning



UMD DATA605 - Big Data Systems

07.2: Data Wrangling and Cleaning

- **Instructor:** Dr. GP Saggese,
- @Reference@
 - Pandas tutorial
 - Web
 - Many free resources (e.g., official docs)
 - Mastery
 - <https://wesmckinney.com/book>
 - Read cover-to-cover and try the examples 2–3 times to really *master* them

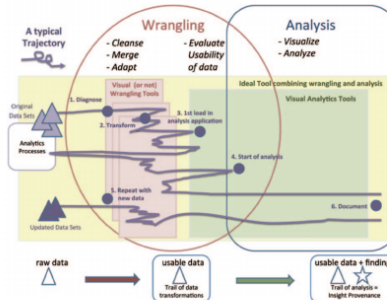


1 / 15

2 / 15: Data Wrangling and Data Cleaning

Data Wrangling and Data Cleaning

- **Data wrangling and cleaning**
 - Turn *messy, inconsistent, incomplete* data into a *coherent* dataset
- **Most effort** in data science is spent here, not on modeling
 - It is iterative work
 - Clean → Model → Clean → ...
- **Data wrangling**
 - Transform data into a useful structure
 - Reshape tables, merge sources, create new variables
- **Data cleaning**
 - Correct or remove incorrect, corrupt, or irrelevant data
 - Handle missing values, outliers, inconsistent formats
- **Often done together**



2 / 15

- **Data wrangling and cleaning**
 - The goal is to turn *messy, inconsistent, and incomplete* data into a *coherent* dataset that can be easily analyzed.
 - This process is crucial because most of the effort in data science is spent here rather than on modeling. It involves an iterative cycle: Clean → Model → Clean → ...
- **Data wrangling**
 - This involves transforming data into a useful structure. It includes reshaping tables, merging different data sources, and creating new variables to better suit analysis needs.
- **Data cleaning**
 - This step focuses on correcting or removing incorrect, corrupt, or irrelevant data. It involves handling missing values, identifying and managing outliers, and ensuring consistent data formats.
- **Often done together**
 - Data wrangling and cleaning are typically performed together as they complement each other in preparing data for analysis. The process is visualized as a trajectory from raw data to usable data, and finally to insights, highlighting the importance of both wrangling and analysis.

3 / 15: Typical Data Wrangling Workflow

Typical Data Wrangling Workflow

- **Inspect data**
 - Look at column names, data types
 - Look at a few rows
 - Basic summaries
- **Diagnose problems**
 - Missing values, unusual codes, inconsistent units, strange distributions
- **Decide strategy**, based on project
 - What to keep
 - What to fix
 - What to drop
- **Apply transformations**
 - Filter, join, reshape, recode, aggregate
- **Validate results**
 - Check that changes make sense and that key statistics are reasonable



3 / 15

- **Inspect data**
 - *Look at column names, data types:* Begin by examining the structure of your dataset. Understanding the column names and data types helps in identifying what kind of data you are dealing with.
 - *Look at a few rows:* Reviewing a sample of the data gives a quick sense of its content and potential issues.
 - *Basic summaries:* Generate summaries like mean, median, and mode to get an overview of the data distribution.
- **Diagnose problems**
 - *Missing values, unusual codes, inconsistent units, strange distributions:* Identify common data issues such as missing entries, unexpected codes, or inconsistent measurement units. Recognizing these problems early is crucial for effective data cleaning.
- **Decide strategy, based on project**
 - *What to keep:* Determine which data is essential for your analysis.
 - *What to fix:* Decide on methods to correct or impute problematic data.
 - *What to drop:* Identify and remove data that is irrelevant or too problematic to fix.
- **Apply transformations**
 - *Filter, join, reshape, recode, aggregate:* Use various data manipulation techniques to prepare the data for analysis. This might involve filtering out unnecessary data, joining datasets, reshaping data structures, recoding values, or aggregating data for summary statistics.
- **Validate results**
 - *Check that changes make sense and that key statistics are reasonable:* After transformations, ensure that the data still makes logical sense and that key statistics align with

expectations. This step is crucial to confirm that the data is ready for analysis.

4 / 15: Data Extraction

Data Extraction

- **Web scraping**
 - Use APIs or scrape data explicitly
 - Scraping is challenging
 - Fragile (page changes can break your parser)
 - Throttling/rate limits
 - Cat-and-mouse game with websites
 - Set up pipelines for periodic scraping
 - Tools for automated scraping
 - E.g., [import.io](#), [portia](#), ..
- **Various sources of data**
 - Files (e.g., CSV, JSON, XML)
 - Databases
 - Spreadsheets
 - AWS S3 buckets
 - Web APIs
 - Logs
 - ...
- Raw data reflects **collection needs, not analysis needs**

4 / 15

- **Data Extraction**
- **Web scraping**
 - *Web scraping* involves collecting data from websites. You can either use *APIs* provided by the website or directly scrape the data from the web pages. APIs are often more stable and reliable, but not all websites offer them.
 - Scraping can be tricky because websites frequently update their layouts, which can break your scraping scripts. This is why scraping is described as *fragile*.
 - Websites may impose *throttling* or *rate limits* to prevent excessive access, which can slow down your data collection process.
 - Scraping often turns into a *cat-and-mouse game* where websites try to block scrapers, and scrapers try to bypass these blocks.
 - To manage ongoing data collection, it's important to set up *pipelines* that automate the scraping process at regular intervals.
 - There are several tools available to help automate web scraping, such as [import.io](#) and [portia](#), which can simplify the process.
- **Various sources of data**
 - Data can come from many different places, including files like CSV, JSON, and XML, which are common formats for storing structured data.
 - Databases are another major source, often used for storing large amounts of data in a structured way.

- Spreadsheets are widely used for smaller datasets and are easy to manipulate manually.
- Cloud storage solutions like AWS S3 buckets can hold vast amounts of data and are often used for backup and sharing.
- Web APIs provide a way to access data programmatically from web services.
- Logs, which record events or transactions, can be a rich source of data for analysis.
- **Raw data reflects collection needs, not analysis needs**
 - The data you collect is often in its *raw form*, meaning it is structured to meet the needs of the collection process rather than being ready for analysis. This means that before you can analyze the data, you may need to clean and transform it to fit your specific analysis requirements.

5 / 15: Tidy Data

Tidy Data

- **Definition:** Tidy data is a structured format for datasets that makes them easier to manipulate, analyze, and visualize
 - Tidy data, Wickham, 2014
- **Core Principles**
 - Each variable forms a *column*
 - Each observation forms a *row*
 - Each type of observational unit forms a *table*
 - Key tools in pandas: `pandas.melt()`, `pandas.pivot()`

	treatmenta	treatmentb
John Smith	—	2
Jane Doe	16	11
Mary Johnson	3	1

	John Smith	Jane Doe	Mary Johnson
treatmenta	—	16	3
treatmentb	2	11	1

“Messy” data

name	trt	result
John Smith	a	—
Jane Doe	a	16
Mary Johnson	a	3
John Smith	b	2
Jane Doe	b	11
Mary Johnson	b	1

Tidy data

5 / 15

- **Definition:** Tidy data is a way of organizing datasets to make them easier to work with. This format helps in data manipulation, analysis, and visualization. The concept was popularized by Hadley Wickham in 2014.
- **Core Principles:**
 - Each *variable* should be in its own column. This means that every piece of information that describes a characteristic of the data should have a dedicated column.
 - Each *observation* should be in its own row. An observation is a single data point or record.
 - Each type of *observational unit* should be in its own table. This means that different kinds of data should be separated into different tables.
 - In Python’s pandas library, tools like `pandas.melt()` and `pandas.pivot()` are essential

for reshaping data into a tidy format.

– **Visuals:**

- The images illustrate the difference between “messy” and tidy data. Messy data can have variables and observations mixed up, making analysis difficult.
- The tidy data example shows a clear structure where each row is an observation, and each column is a variable, making it straightforward to analyze and visualize.

6 / 15: Reshaping data

Reshaping data

- **Wide to long**
 - Turn similar columns into key-value pairs
- **Long to wide**
 - Turn key-value pairs into separate columns
- **What is the best shape?**
 - Choose the shape that simplifies analysis and visualization

	type	date	clicks	conversions	impressions
0		2020-01-01	1.0	NaN	18.0
1		2020-01-02	2.0	NaN	19.0
2		2020-01-03	1.0	1.0	14.0
3		2020-01-04	NaN	NaN	5.0
4		2020-01-05	1.0	NaN	8.0
5		2020-01-06	1.0	1.0	15.0
6		2020-01-07	2.0	NaN	8.0

Wide format

	date	type	count
0	2020-01-01	impressions	18.0
1	2020-01-02	impressions	19.0
2	2020-01-03	impressions	14.0
3	2020-01-04	impressions	5.0
4	2020-01-05	impressions	8.0
...	...	...	...
91	2020-01-28	conversions	NaN
92	2020-01-29	conversions	NaN
93	2020-01-30	conversions	NaN
94	2020-01-31	conversions	NaN
95	2020-02-01	conversions	NaN

Long format

6 / 15

– **Reshaping data**

- *Wide to long*: This process involves transforming data from a wide format, where similar data points are spread across multiple columns, into a long format. In the long format, these data points are consolidated into key-value pairs, making it easier to perform operations like grouping and summarizing.
- *Long to wide*: This is the reverse process, where data in a long format with key-value pairs is spread out into separate columns. This format can be more intuitive for certain types of analysis and visualization, especially when comparing multiple variables side by side.
- *What is the best shape?:* The choice between wide and long formats depends on the specific analysis or visualization task. A shape that simplifies the process and makes the data easier to understand is generally preferred. For example, wide format might be better for visualizing trends over time, while long format is often more suitable for statistical analysis.

– **Images Explanation**

- The first image shows data in a wide format, where each type of data (clicks, conversions, impressions) has its own column.
- The second image displays the same data in a long format, where each row represents a single observation with a type and count, making it easier to perform operations like filtering and aggregation.

7 / 15: Split-Apply-Combine

Split-Apply-Combine

- **Common data analysis pattern**
 - *Split*: break data into smaller pieces
 - *Apply*: operate on each piece independently
 - *Combine*: combine pieces back together
 - The Split-Apply-Combine Strategy for Data Analysis, Wickam, 2011
- **Examples**
 - Group-wise ranking
 - Group statistics (sums, means, counts)
 - Create separate models per group
- **Pros**
 - Code is compact
 - Easy to parallelize
- **Supported by many systems**
 - Pandas
 - **SQL GROUP BY** operator
 - Map-Reduce

```
In [94]: animals.groupby("kind").height.agg(
...:     min_height="min",
...:     max_height="max",
...: )
Out[94]:
```

	min_height	max_height
kind		
cat	9.1	9.5
dog	6.0	34.0

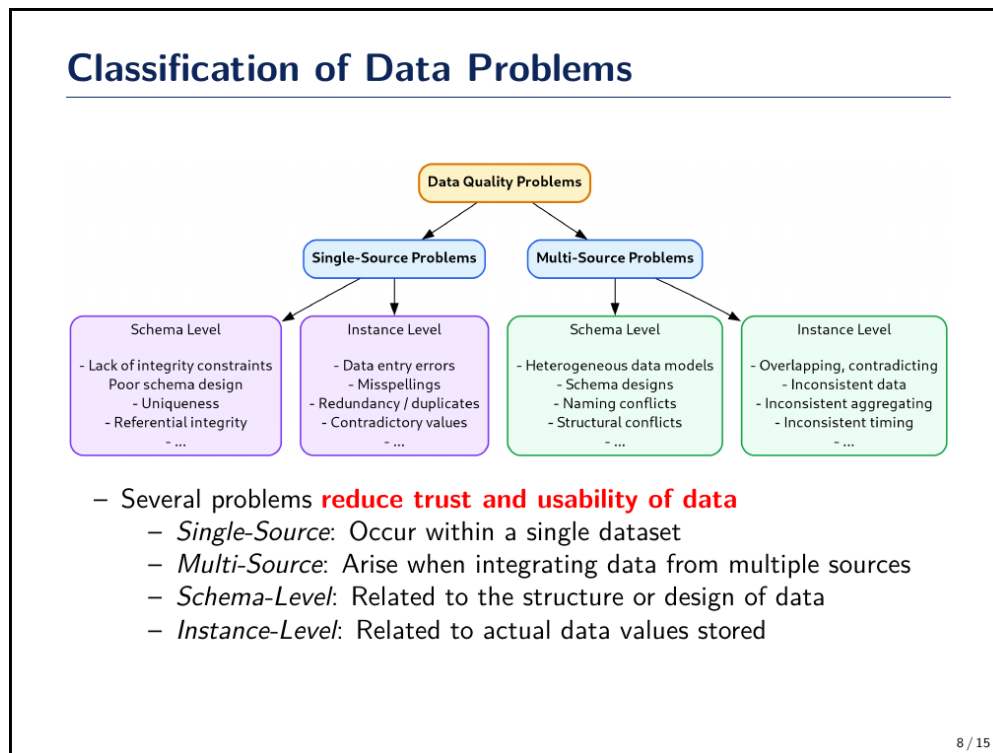
7 / 15

- **Split-Apply-Combine**
 - This is a *common data analysis pattern* used to simplify complex data operations.
 - **Split**: The data is divided into smaller, more manageable pieces based on some criteria. For example, splitting a dataset by categories like “kind” or “type.”
 - **Apply**: Each piece is processed independently. This could involve calculations, transformations, or any operation needed on the data.
 - **Combine**: The results from each piece are brought back together into a single dataset. This step ensures that the analysis is cohesive and complete.
 - The concept is well-documented in *The Split-Apply-Combine Strategy for Data Analysis* by Wickham (2011).
- **Examples**
 - *Group-wise ranking*: Sorting or ranking data within each group.
 - *Group statistics*: Calculating sums, means, or counts for each group.
 - *Create separate models per group*: Building individual models for each subset of data.
- **Pros**
 - The code is *compact*, making it easier to write and understand.
 - It is *easy to parallelize*, allowing for efficient processing of large datasets.
- **Supported by many systems**

- **Pandas:** A popular Python library for data manipulation.
- **SQL GROUP BY** operator: Used in SQL databases to group data.
- **Map-Reduce:** A programming model for processing large datasets with a distributed algorithm.

The code example demonstrates using Pandas to group data by “kind” and calculate the minimum and maximum heights for each group, showcasing the practical application of the Split-Apply-Combine pattern.

8 / 15: Classification of Data Problems



- **Data Quality Problems**
 - Data quality issues can significantly reduce the trust and usability of data. These problems are broadly classified into two categories: *Single-Source Problems* and *Multi-Source Problems*.
- **Single-Source Problems**
 - These occur within a single dataset and can be further divided into:
 - **Schema Level:** Issues related to the structure or design of the data. Common problems include:
 - Lack of integrity constraints, which can lead to poor schema design.
 - Issues with uniqueness and referential integrity.
 - **Instance Level:** Problems related to the actual data values stored. Examples include:
 - Data entry errors and misspellings.
 - Redundancy or duplicates and contradictory values.
- **Multi-Source Problems**
 - These arise when integrating data from multiple sources and are also divided into:

- **Schema Level:** Challenges due to differences in data models and schema designs. This includes:
 - Naming conflicts and structural conflicts.
- **Instance Level:** Issues with the data values themselves, such as:
 - Overlapping, contradicting, and inconsistent data.
 - Problems with inconsistent aggregating and timing.

Understanding these classifications helps in identifying and addressing data quality issues effectively.

9 / 15: Single-Source Problems

Single-Source Problems

- **Depends largely on source**
 - *Databases* often enforce schema / instance constraints
 - *Spreadsheet data* is often “clean”
 - Schema exists
 - *Logs* are messy
 - *Web-page data* is messier
- **Types of problems**
 - Ill-formatted data
 - Missing/illegal values, misspellings, wrong fields, extraction issues
 - Duplicated records, contradicting information, referential integrity violations
 - Unclear default/missing values
 - Evolving schemas/classification schemes (categorical attributes)
 - Outliers

9 / 15

- **Single-Source Problems**
- **Depends largely on source**
 - When dealing with data from a *single source*, the quality and structure of the data can vary significantly depending on where it comes from.
 - **Databases:** These are typically well-organized because they enforce rules about how data should be stored, known as schema and instance constraints. This means the data is usually consistent and reliable.
 - **Spreadsheet data:** This type of data is often considered “clean” because it usually has a defined structure or schema, making it easier to work with.
 - **Logs:** These are records of events or transactions and can be quite messy. They might not follow a strict format, making them harder to analyze.
 - **Web-page data:** This is often even messier than logs because web pages are designed for human readability, not machine processing, leading to inconsistencies and irregularities.
- **Types of problems**

- **Ill-formatted data:** This refers to data that doesn't follow a consistent format, making it difficult to process or analyze.
- **Missing/illegal values, misspellings, wrong fields, extraction issues:** These are common issues where data might be incomplete, contain errors, or be placed in the wrong category, complicating data analysis.
- **Duplicated records, contradicting information, referential integrity violations:** These problems occur when data is repeated unnecessarily, contains conflicting information, or fails to maintain relationships between different data points.
- **Unclear default/missing values:** Sometimes, it's not clear what a missing value should be, or default values are not well-defined, leading to confusion.
- **Evolving schemas/classification schemes (categorical attributes):** As data evolves, the way it is categorized or structured might change, which can cause inconsistencies over time.
- **Outliers:** These are data points that are significantly different from others, which can skew analysis and lead to incorrect conclusions.

10 / 15: Single-Source Problems: Example

Single-Source Problems: Example

- **Sources of errors in data**
 - Data entry errors: arbitrary values entered
 - Measurement errors: sensor data inaccuracies
 - Distillation errors: processing and summarization issues
 - Data integration errors: inconsistencies across combined sources

Scope/Problem		Dirty Data	Reasons/Remarks
Attribute	Missing values	phone=9999-999999	unavailable values during data entry (dummy values or null)
	Misspellings	city="Liipzig"	usually typos, phonetic errors
	Cryptic values, Abbreviations	experience="B"; occupation="DB Prog."	
	Embedded values	name="J. Smith 12.02.70 New York"	multiple values entered in one attribute (e.g. in a free-form field)
	Misfielded values	city="Germany"	
Record	Violated attribute dependencies	city="Redmond", zip=77777	city and zip code should correspond
Record type	Word transpositions	name,= "J. Smith", name,="Miller P."	usually in a free-form field
	Duplicated records	emp,=(name="John Smith",...); emp,=(name="J. Smith",...)	same employee represented twice due to some data entry errors
	Contradicting records	emp,=(name="John Smith", bdate=12.02.70); emp,=(name="John Smith", bdate=12.12.70)	the same real world entity is described by different values
Source	Wrong references	emp=(name="John Smith", deptno=17)	referenced department (17) is defined but wrong

Table 2. Examples for single-source problems at instance level

10 / 15

- **Sources of errors in data**
 - **Data entry errors:** These occur when incorrect or arbitrary values are entered into the system. For example, a phone number might be entered as "9999-999999" when the actual number is unavailable. This often happens due to the use of placeholder or dummy values.
 - **Measurement errors:** These are inaccuracies that arise from the tools or sensors used to collect data. Such errors can lead to incorrect data being recorded.
 - **Distillation errors:** These occur during the processing and summarization of data.

Errors in this stage can lead to incorrect conclusions or summaries being drawn from the data.

- **Data integration errors:** These happen when data from different sources are combined, leading to inconsistencies. For example, mismatched city and zip code data can occur if integration is not handled properly.
- **Table Explanation**
 - **Attribute Level:** Issues like missing values, misspellings, and embedded values can lead to incorrect data interpretation. For instance, “city=‘Liepzig’” is likely a typo for “Leipzig.”
 - **Record Level:** Violated attribute dependencies, such as mismatched city and zip codes, can cause data integrity issues.
 - **Record Type Level:** Problems like word transpositions and duplicated records can result in multiple entries for the same entity, complicating data analysis.
 - **Source Level:** Wrong references, such as incorrect department numbers, can lead to misclassification and errors in data reporting.

11 / 15: Multi-Source Problems

Multi-Source Problems

- **Different data sources**
 - Developed separately
 - Maintained by different people
 - Stored in different systems
- **Schema mapping / transformation**
 - Map information across sources
 - Naming conflicts
 - Same name for different objects
 - Different names for same objects
 - Different representations across sources
- **Entity resolution**
 - Match entities across sources
- **Data quality issues**
 - Contradicting information
 - Mismatched information

11 / 15

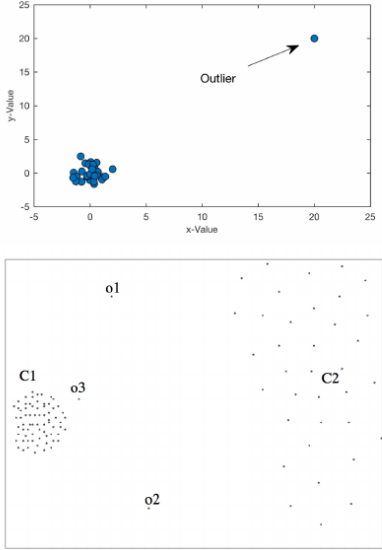
- **Multi-Source Problems**
 - When dealing with *different data sources*, it’s important to understand that these sources often come from various origins. They might have been developed separately, which means they were created with different goals or standards in mind. Additionally, these sources are usually maintained by different people, leading to variations in how data is updated or corrected. Furthermore, they are stored in different systems, which can complicate efforts to integrate or compare data.
- **Schema mapping / transformation**

- This process involves mapping information across different data sources to create a unified view. One common challenge is *naming conflicts*. Sometimes, the same name might be used for different objects, causing confusion. Conversely, different names might be used for the same object, making it hard to recognize that they refer to the same thing. Additionally, data might be represented differently across sources, requiring transformation to ensure consistency.
- **Entity resolution**
 - This is the task of matching entities across different data sources. It involves identifying records that refer to the same real-world entity, even if they appear differently in each source. This step is crucial for creating a comprehensive and accurate dataset.
- **Data quality issues**
 - When integrating data from multiple sources, you might encounter contradicting information, where different sources provide conflicting data about the same entity. There can also be mismatched information, where data doesn't align correctly across sources. Addressing these issues is essential for maintaining the integrity and reliability of the combined dataset.

12 / 15: Univariate Outlier Detection

Univariate Outlier Detection

- **Problems with outliers**
 - Extreme outliers may alter metrics, masking others
- Use statistics to **identify outliers**
 - Robust statistics: minimize effect of corrupted data
 - Robust center metrics
 - Median
 - $k\%$ -trimmed mean (discard lowest and highest $k\%$ values)
 - Robust dispersion
 - Median absolute deviation (MAD)
 - Median distance from median value



12 / 15

- **Problems with outliers**
 - Outliers are data points that are significantly different from others. They can skew results and affect the accuracy of statistical analyses. Extreme outliers can particularly distort metrics, making it difficult to identify patterns or trends in the data.
- **Use statistics to identify outliers**
 - *Robust statistics* are used to minimize the impact of outliers. These methods are less sensitive to extreme values and provide a more accurate representation of the data.

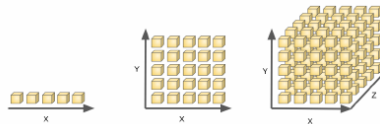
- **Robust center metrics**
 - *Median*: The middle value of a dataset, which is not affected by extreme values.
 - *k%-trimmed mean*: This involves removing the lowest and highest k% of values, reducing the influence of outliers.
- **Robust dispersion**
 - *Median absolute deviation (MAD)*: Measures variability by calculating the median of absolute deviations from the median, offering a robust alternative to standard deviation.
 - *Median distance from median value*: Another method to assess dispersion, focusing on the median to reduce outlier impact.

These techniques help in accurately identifying and managing outliers, ensuring that data analysis remains reliable and meaningful.

13 / 15: Outlier Detection

Outlier Detection

- **For Gaussian data**
 - Data points 1.5x MAD from median
 - Plot histogram to confirm
- **For non-Gaussian data**
 - Estimate generating distribution (parametric or not)
 - Distance-based: find points with few neighbors
 - Density-based:
 - Define *density* as the average distance to k nearest neighbors
 - Use relative density to identify outliers
- **Curse of dimensionality**
 - “In high-dimensional spaces, data is sparse”
 - Techniques break down as data dimensionality increases
 - Need $O(e^n)$ points with n dimensions to estimate
 - Project data into lower-dimensional space to find outliers



13 / 15

Outlier Detection

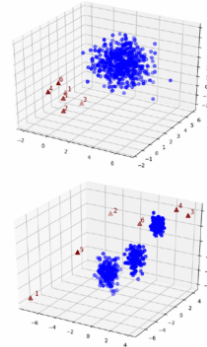
- **For Gaussian data**
 - When dealing with Gaussian (normal) data, outliers can be identified by looking at data points that are 1.5 times the Median Absolute Deviation (MAD) from the median. This method helps in pinpointing values that deviate significantly from the central tendency.
 - It's useful to plot a histogram of the data to visually confirm the presence of outliers. This visual check can help ensure that the statistical method aligns with the data's distribution.
- **For non-Gaussian data**
 - For data that doesn't follow a Gaussian distribution, it's important to estimate the distri-

- bution that generated the data. This can be done using parametric methods (assuming a specific distribution) or non-parametric methods (making fewer assumptions).
- **Distance-based methods:** Identify outliers by finding data points that have few neighbors, indicating they are isolated from the rest of the data.
- **Density-based methods:** Define *density* as the average distance to the k nearest neighbors. Outliers are identified by their relative density compared to other points, highlighting those that are in less dense regions.
- **Curse of dimensionality**
 - In high-dimensional spaces, data becomes sparse, meaning that data points are spread out and isolated. This sparsity makes it difficult for traditional techniques to work effectively.
 - As the number of dimensions increases, the number of data points needed to maintain the same density grows exponentially, denoted as $O(e^n)$ for n dimensions.
 - To address this, data can be projected into a lower-dimensional space to better identify outliers. However, this process is not straightforward and requires careful consideration to preserve the data's essential characteristics.

14 / 15: Multivariate Outliers

Multivariate Outliers

- **Mean/covariance not robust**
 - Sensitive to outliers
 - Use robust statistics
- Assume **multivariate Gaussian data**
 - Defined by mean $\underline{\mu}$ and covariance matrix $\underline{\Sigma}$
 - *Iterative approach*
 - Mahalanobis distance:
$$\sqrt{(\underline{x} - \underline{\mu})^T \underline{\Sigma}^{-1} (\underline{x} - \underline{\mu})}$$
 - Measures distance of point $\underline{x}$ from distribution
 - Identify outliers: points too far by Mahalanobis distance
 - Remove outliers
 - Recompute mean and covariance
- Data volume often large
 - Use approximation techniques
- Try different techniques based on data



14 / 15

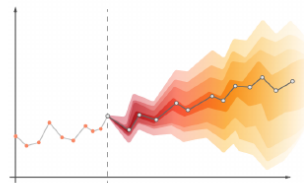
- **Mean/covariance not robust**
 - Traditional mean and covariance calculations are *sensitive to outliers*. This means that a few extreme values can significantly skew these statistics, leading to inaccurate representations of the data.
 - To address this, it's important to use *robust statistics* that are less affected by outliers, ensuring a more accurate analysis.
- **Assume multivariate Gaussian data**

- Multivariate Gaussian data is characterized by a *mean vector* ($\underline{\mu}$) and a *covariance matrix* ($\underline{\Sigma}$). These parameters define the center and spread of the data in multiple dimensions.
- An *iterative approach* is often used to handle outliers:
 - The *Mahalanobis distance* is a key tool. It measures how far a point $\underline{x}$ is from the mean of the distribution, taking into account the covariance.
 - Points with a large Mahalanobis distance are considered outliers. These are removed to prevent them from skewing the data analysis.
 - After removing outliers, the mean and covariance are recalculated to refine the data model.
- **Data volume often large**
 - With large datasets, computational efficiency becomes crucial. *Approximation techniques* can be used to manage the data size and complexity, making the analysis feasible.
- **Try different techniques based on data**
 - It's important to experiment with various methods to find the most effective approach for identifying and handling outliers, as different datasets may require different strategies.

15 / 15: Time Series Outliers

Time Series Outliers

- Often data is in the form of a **time series**
- A **time series** is a sequence of data points recorded at regular intervals
 - Stock prices
 - Sales revenue
 - Website traffic
 - Inventory levels
 - Energy consumption
 - Market demand
 - Social media engagement
 - Hourly energy usage
 - Weekly retail foot traffic
 - ...
- Rich literature on **forecasting** in time series data
- Use historical patterns to flag **outliers**
 - Rolling MAD (median absolute variation)



15 / 15

- **Time Series Data**
 - A *time series* is a collection of data points gathered over consistent time intervals. This type of data is crucial for analyzing trends and patterns over time.
 - Examples include:
 - Stock prices: Daily or minute-by-minute changes in the stock market.
 - Sales revenue: Monthly or quarterly sales figures for a company.
 - Website traffic: Hourly or daily visits to a website.
 - Inventory levels: Regular checks on stock quantities.

- Energy consumption: Monitoring electricity usage over time.
- Market demand: Tracking consumer interest in products.
- Social media engagement: Measuring interactions on platforms.
- Hourly energy usage: Detailed tracking of energy consumption.
- Weekly retail foot traffic: Counting visitors to a store.
- **Forecasting in Time Series**
 - There is extensive research on predicting future values in time series data, known as *forecasting*. This involves using past data to make informed predictions about future trends.
- **Identifying Outliers**
 - Outliers are data points that deviate significantly from the expected pattern. Detecting these is crucial for accurate analysis.
 - One method to identify outliers is using the *Rolling MAD (Median Absolute Deviation)*, which helps highlight anomalies by comparing each data point to the median of its neighbors.